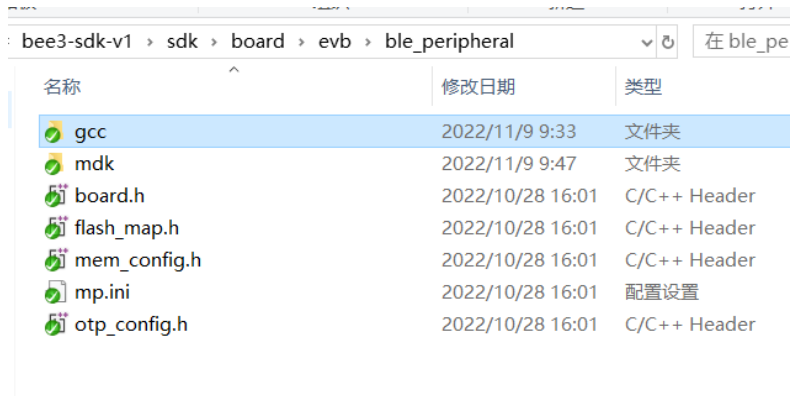


How to create a GCC Project for RTL8762E

This document introduces how to convert a keil project to a GCC project, and demonstrate the generation of app bin files.

1、 Create the “gcc” folder for GCC project and copy the 2 files to it:

- a) app.ld (from gcc project template “sdk\board\evb\ble_perpheral\gcc”)
- b) Makefile (from gcc project template “sdk\board\evb\ble_perpheral\gcc”)



2、 Modify parameters of the makefile

- a) TARGET: project name

```
11
12 #####
13 # target
14 #####
15 TARGET = peripheral
16 ENCRYPTION = 0
```

b) C_SOURCES: the source Files

```

32 BIN_DIR = bin
33 #####
34 # source
35 #####
36 # C sources
37 C_SOURCES = \
38 ../../../../../sdk/src/mcu/rtl876x/overlay_mgr.c \
39 ../../../../../sdk/src/sample/ble_peripheral/main.c \
40 ../../../../../sdk/src/sample/ble_peripheral/peripheral_app.c \
41 ../../../../../sdk/src/sample/ble_peripheral/app_task.c \
42 ../../../../../sdk/src/sample/ble_peripheral/ancs.c \
43 ../../../../../sdk/src/ble/profile/client/ancs_client.c \
44 ../../../../../sdk/src/ble/profile/server/simple_ble_service.c \
45 ../../../../../sdk/src/ble/profile/server/bas.c \
46 ../../../../../sdk/src/mcu/peripheral/rtl876x_io_dlps.c \
47 ../../../../../sdk/src/mcu/rtl876x/system_rtl8762c.c
48

```

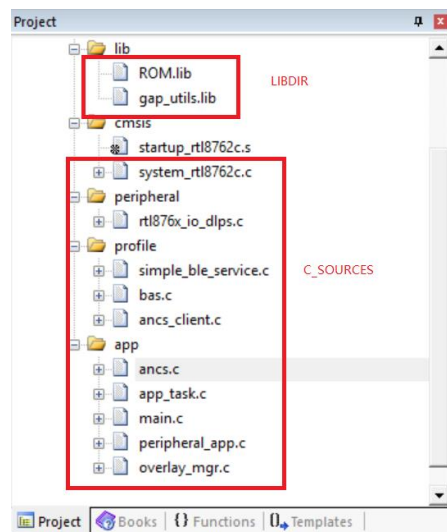
c) LIBDIR: GCC format libs (with file postfix .a)

```

144
145 # libraries
146 LIBS = -lc -lm -lnosys
147 LIBDIR = ../../../../../../sdk/bin/rom_symbol_gcc.axf \
148          ../../../../../../sdk/bin/libgap_lib.a \
149

```

The C source files and libs can be found in the Keil project:



d) C_INCLUDES:

```

149
C_INCLUDES = \
-I../../../../bin/upperstack_img/upperstack_0_0 \
-I../../../../inc/app \
-I../../../../inc/bluetooth/gap \
-I../../../../inc/bluetooth/gap/gap_lib \
-I../../../../inc/bluetooth/profile \
-I../../../../inc/bluetooth/profile/server \
-I../../../../inc/bluetooth/profile/client \
-I../../../../inc/os \
-I../../../../inc/peripheral \
-I../../../../inc/platform \
-I../../../../src/sample/ble_peripheral \
-I../../../../ble_peripheral \
# includes END

```

[illegible]

```

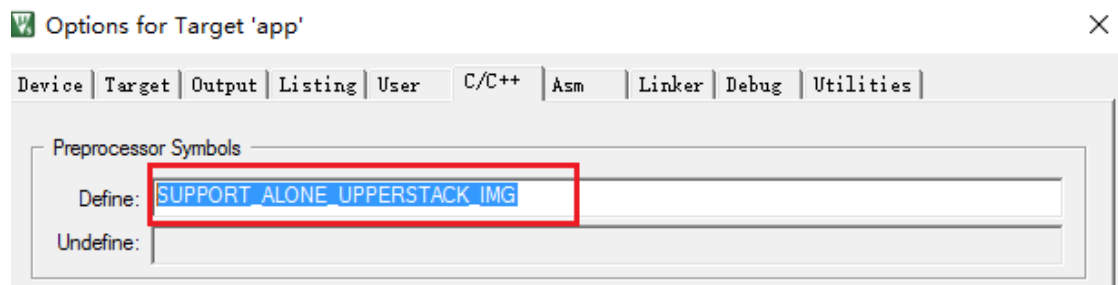
#PER_INCLUDE= \
#include app_define.h \
#include mem_config.h \
#PRE_INCLUDES END

```

f) PRE_DEFINE:

```
PER_DEFINE= \
-D SUPPORT_ALONE_UPPERSTACK_IMG \
-D APP_OS \
#PER_DEFINE END
```

The pre define can be found in the keil project:



g) OPT: O1 is recommend.

```
19 #####
20 # building variables
21 #####
22 # debug build?
23 DEBUG = 1
24 # optimization
25 #OPT = -Og
26 OPT = -O1
27
```

3、 Make

```
D:\code_base\bee3-sdk-v1\sdk\board\evb\ble_peripheral\gcc>mingw32-make
./../../../../tool/memory_icf/MemDefine ../ gcc ./
mkdir build
arm-none-eabi-gcc -c -mcpu=cortex-m0plus -mthumb -mfloat-abi=soft -specs=nano.specs -I../../../../bin/upperstack_img/u
pperstack_0_0 -I../../../../inc/app -I../../../../inc/bluetooth/gap -I../../../../inc/bluetooth/gap_lib -I../../../../
inc/bluetooth/profile -I../../../../inc/bluetooth/profile/server -I../../../../inc/bluetooth/profile/client -I../../../../
inc/os -I../../../../inc/peripheral -I../../../../inc/platform -I../../../../src/sample/ble_peripheral -I../../../../ble p
eripheral -O1 -Wall -fdata-sections -ffunction-sections -std=c99 -MMD -MP -MF build/system_rt1876x.d -include app_fla
gs.h -D SUPPORT_ALONE_UPPERSTACK_IMG -Wa,-a,-ad,-alms=build/system_rt1876x.lst ../../../../../../src/mcu/rt1876x/system_rt18
76x.c -o build/system_rt1876x.o
```

After make, the app bin file and other related files will be generated. You can find the files in the bin folder.

If your environment is windows with MinGW, the default command for compile is "mingw32-make",
If your environment is linux, the default command for compile is "make"

sdk > board > evb > ble_peripheral > gcc > bin				在 bin 中搜索
名称	修改日期	类型	大小	
app.bin	2022/11/9 9:31	FTE Binary Expor...	14 KB	
app.dis	2022/11/9 9:31	DIS 文件	281 KB	
app.trace	2022/11/9 9:31	TRACE 文件	4 KB	
app_MP.bin	2022/11/9 9:31	FTE Binary Expor...	14 KB	
app_MP_sdk_1.1.0.3_1201a39e-9cb6...	2022/11/9 9:31	FTE Binary Expor...	14 KB	

If you want to rebuild the app, you can do “make clean”(or “mingw32-make clean”) first.

```
D:\code_base\bee3-sdk-v1\sdk\board\evb\ble_peripheral\gcc>mingw32-make clean
rm -fR build
rm -fR bin
rm mem_define.ld
```

Note:

The version of linker scripts supports “#define FEATURE_RAM_CODE 0” only, which means the code without any section will run in flash. If you want to put some code in ram, you can add DATA_RAM_FUNCTION like this:

```
void DLPS_IO_EnterDlpsCb(void) DATA_RAM_FUNCTION:
void DLPS_IO_EnterDlpsCb(void)
{
```

To improve the performance of flash code, flash cache can be enabled (set the macro “SHARE_CACHE_RAM_SIZE” in mem_config.h). There are three kinds of size can be set:

1. Flash cache 16 KB (which means no space can be used for data ram)

```
/** @brief shared cache ram size (adjustable, config SHARE_CACHE_RAM_SIZE: 0/8KB/16KB) */
#define SHARE_CACHE_RAM_SIZE (0 * 1024)
```

2. Flash cache 8 KB (which means the left 8 KB can be used for data ram)
3. Flash cache 0 KB (which means the whole 16 KB can be used for data ram)